

UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS

UFR des Sciences Fondamentales et de l'Ingénierie
Département de Mathématiques-Informatique

Travaux Dirigés Programmation Arduino

Thème : Création et Modification de Bibliothèques

Enseignant : Dr B. DIOP

Objectif : Maîtriser la création de bibliothèques Arduino en appliquant les concepts de Programmation Orientée Objet, tout en intégrant des algorithmes de gestion de données.

Partie I : Analyse et Compréhension de Bibliothèques

Exercice 1. Anatomie d'une bibliothèque

On vous donne la structure de fichiers suivante :

```
MaLibrairie/  
  MaLibrairie.h  
  MaLibrairie.cpp  
  keywords.txt  
  examples/  
    ExempleBasique/  
      ExempleBasique.ino
```

Questions :

1. Quel est le rôle de chaque fichier dans cette structure ?
2. Pourquoi le fichier `.h` est-il appelé "fichier d'en-tête" ?
3. Où doit-on placer ce dossier pour que l'IDE Arduino le reconnaisse ?
4. Que se passe-t-il si on oublie `#include "Arduino.h"` dans le fichier `.cpp` ?

Exercice 2. Les Include Guards

Soit le fichier d'en-tête suivant, incomplet :

```
1 // Fichier : Capteur.h  
2  
3 // ??? A COMPLETER ???  
4  
5 #include "Arduino.h"
```

```
6
7 class Capteur {
8 public:
9     Capteur(int broche);
10    int lire();
11 private:
12    int _broche;
13 };
14
15 // ??? A COMPLETER ???
```

Travail demandé :

1. Compléter le fichier avec les *include guards* appropriés.
2. Expliquer pourquoi ces directives sont indispensables.
3. Que signifient les directives `#ifndef`, `#define` et `#endif` ?

Exercice 3. Public vs Private

Soit la classe suivante :

```
1 class MoteurDC {
2 public:
3     MoteurDC(int pinA, int pinB, int pinPWM);
4     void avancer(int vitesse);
5     void reculer(int vitesse);
6     void arreter();
7     int getVitesse();
8
9 private:
10    int _pinA, _pinB, _pinPWM;
11    int _vitesseActuelle;
12    void _appliquerPWM(int valeur);
13 };
```

Questions :

1. Parmi les éléments suivants, lesquels sont accessibles depuis un croquis `.ino` ?
 - `monMoteur.avancer(200);`
 - `monMoteur._pinA = 5;`
 - `monMoteur.getVitesse();`
 - `monMoteur._appliquerPWM(100);`
2. Pourquoi les broches matérielles sont-elles placées en `private` ?
3. Quel est l'intérêt du préfixe `_` (underscore) devant les variables privées ?

Exercice 4. L'opérateur de résolution de portée

On vous donne le fichier `LED.h` :

```
1 class LED {
2 public:
3     LED(int broche);
4     void allumer();
5     void eteindre();
```

```
6     void basculer();
7     bool estAllumee();
8 private:
9     int _broche;
10    bool _etat;
11 };
```

Travail demandé : Écrire le fichier `LED.cpp` correspondant, en utilisant correctement l'opérateur `::` pour chaque méthode.

Partie II : Création de Bibliothèques Simples

Exercice 5. Bibliothèque Bouton avec anti-rebond

Créer une bibliothèque `Bouton` qui gère un bouton poussoir avec filtrage anti-rebond logiciel.

Spécifications :

- Le constructeur reçoit la broche et le délai anti-rebond (en ms).
- Une méthode `actualiser()` à appeler dans `loop()`.
- Une méthode `estPresse()` retourne `true` si le bouton est enfoncé.
- Une méthode `vientDEtrePresse()` retourne `true` une seule fois par appui.

Travail demandé :

1. Écrire le fichier `Bouton.h` avec la déclaration de la classe.
2. Écrire le fichier `Bouton.cpp` avec l'implémentation.
3. Écrire un croquis d'exemple qui compte les appuis sur le bouton.

Indication : Utiliser `millis()` pour gérer le temps sans bloquer.

Exercice 6. Bibliothèque CapteurAnalogique avec moyenne glissante

Créer une bibliothèque `CapteurAnalogique` qui lit un capteur et calcule automatiquement la moyenne des N dernières lectures pour filtrer le bruit.

Spécifications :

- Le constructeur reçoit la broche analogique et la taille du buffer ($N \leq 20$).
- Une méthode `lire()` effectue une lecture et met à jour le buffer circulaire.
- Une méthode `getMoyenne()` retourne la moyenne des N dernières lectures.
- Une méthode `getMin()` et `getMax()` pour les extremums du buffer.

Travail demandé :

1. Concevoir la structure de données interne (buffer circulaire).
2. Écrire les fichiers `CapteurAnalogique.h` et `CapteurAnalogique.cpp`.
3. Justifier le choix de limiter N à 20 (contrainte mémoire).

Question algorithmique : Comment calculer la moyenne en $O(1)$ à chaque nouvelle lecture, sans parcourir tout le buffer ?

Exercice 7. Bibliothèque Chronometre

Créer une bibliothèque **Chronometre** pour mesurer des durées avec précision.

Spécifications :

- Méthodes : `demarrer()`, `arreter()`, `reset()`, `pause()`, `reprendre()`.
- Méthode `getTempsMs()` retourne le temps écoulé en millisecondes.
- Méthode `getTempsFormate(char* buffer)` écrit le temps au format “MM:SS.mmm”.

Travail demandé :

1. Identifier les variables privées nécessaires (temps de départ, état, temps accumulé...).
2. Écrire la bibliothèque complète.
3. Gérer le cas où `getTempsMs()` est appelé pendant que le chronomètre tourne.

Défi : Gérer le débordement de `millis()` (après ~49 jours).

Exercice 8. Bibliothèque PWMDoux (Fade sans delay)

Créer une bibliothèque **PWMDoux** qui fait varier progressivement la luminosité d’une LED sans utiliser `delay()`.

Spécifications :

- Le constructeur reçoit la broche PWM et la durée d’un cycle complet (ms).
- Méthodes : `fadeIn()`, `fadeOut()`, `fadeInOut()` (boucle continue).
- Une méthode `actualiser()` à appeler dans `loop()`.
- Possibilité de changer la vitesse dynamiquement.

Travail demandé : Écrire la bibliothèque en utilisant le principe du “Blink Without Delay”.

Partie III : Bibliothèques avec Gestion de Tableaux

Exercice 9. Bibliothèque HistoriqueCapteur

Créer une bibliothèque qui mémorise l’historique des N dernières mesures d’un capteur et fournit des statistiques.

Spécifications :

- Stockage en buffer circulaire de taille configurable (max 50 entrées).
- Chaque entrée contient : valeur (int) + timestamp (unsigned long).
- Méthodes statistiques : `getMoyenne()`, `getEcartType()`, `getTendance()`.
- Méthode `getValeur(int i)` pour accéder à la i-ème valeur la plus récente.

Travail demandé :

1. Définir une structure **Mesure** pour stocker valeur + timestamp.
2. Implémenter le buffer circulaire avec gestion d’index.
3. Calculer la tendance (pente de régression linéaire simplifiée).

```

1 // Structure suggeree
2 struct Mesure {
3     int valeur;
4     unsigned long timestamp;
5 };
6
7 class HistoriqueCapteur {
8 public:
9     HistoriqueCapteur(int broche, int tailleMax);
10    void enregistrer();
11    float getMoyenne();
12    float getTendance(); // Positif = croissant, Negatif = décroissant
13    // ...
14 private:
15     Mesure _buffer[50];
16     int _taille, _tailleMax, _index;
17     // ...
18 };

```

Exercice 10. Bibliothèque GestionnaireEvenements

Créer une bibliothèque qui gère une file d'événements à traiter (pattern "Event Queue").

Spécifications :

- File FIFO de 16 événements maximum.
- Chaque événement a un type (byte) et une donnée (int).
- Méthodes : ajouter(type, donnee), retirer(), estVide(), estPleine().
- Méthode compterParType(byte type) retourne le nombre d'événements de ce type.

Travail demandé :

1. Implémenter la file circulaire avec deux index (tête et queue).
2. Gérer les cas de débordement (file pleine) et de sous-débordement (file vide).
3. Écrire un exemple où des interruptions ajoutent des événements et loop() les traite.

```

1 struct Evenement {
2     byte type;
3     int donnee;
4 };
5
6 class GestionnaireEvenements {
7 public:
8     bool ajouter(byte type, int donnee); // false si plein
9     Evenement retirer();
10    bool estVide();
11    // ...
12 private:
13    Evenement _file[16];
14    volatile byte _tete, _queue; // volatile car modifie par ISR
15 };

```

Exercice 11. Bibliothèque `TableauTrie`

Créer une bibliothèque qui maintient un tableau toujours trié, avec insertion et recherche efficaces.

Spécifications :

- Capacité maximale configurable (défaut : 20 éléments).
- Méthode `insérer(int valeur)` insère en gardant l'ordre croissant.
- Méthode `rechercher(int valeur)` utilise la recherche dichotomique.
- Méthode `supprimer(int valeur)` retire la première occurrence.
- Méthodes `getMin()`, `getMax()`, `getMediane()`.

Travail demandé :

1. Implémenter l'insertion avec décalage des éléments.
2. Implémenter la recherche dichotomique (complexité $O(\log n)$).
3. Comparer avec une approche non triée : quand est-ce avantageux ?

Question : Si on insère fréquemment mais qu'on recherche rarement, cette structure est-elle adaptée ?

Exercice 12. Bibliothèque `MatriceLED`

Créer une bibliothèque pour contrôler une matrice LED 8×8 avec buffer d'affichage.

Spécifications :

- Double buffer (front/back) pour éviter le scintillement.
- Méthodes : `setPixel(x, y, etat)`, `clear()`, `fill()`, `swap()`.
- Méthode `afficher()` pour le multiplexage (à appeler très fréquemment).
- Stockage compact : 8 octets par buffer (1 bit par LED).

Travail demandé :

1. Utiliser les opérations bit à bit pour manipuler les pixels.
2. Implémenter le multiplexage ligne par ligne.
3. Stocker des motifs prédéfinis en `PROGMEM`.

```
1 class MatriceLED {
2 public:
3     MatriceLED(/* broches... */);
4     void setPixel(byte x, byte y, bool etat);
5     bool getPixel(byte x, byte y);
6     void afficher(); // Multiplexage d'une ligne
7     void swap();     // Echange front/back
8 private:
9     byte _front[8];
10    byte _back[8];
11    byte _ligneActuelle;
12    // ...
13};
```

Partie IV : Bibliothèques Avancées avec POO

Exercice 13. Héritage : Capteurs spécialisés

On dispose d'une classe de base `CapteurBase` :

```
1 class CapteurBase {
2 public:
3     CapteurBase(int broche);
4     virtual int lireBrut();           // Lecture ADC brute
5     virtual float lireUnite();       // A redefinir dans les classes filles
6     void setBroche(int broche);
7 protected:
8     int _broche;
9 };
```

Travail demandé : Créer deux classes filles qui héritent de `CapteurBase` :

1. `CapteurTemperature` : convertit la lecture en degrés Celsius (formule du LM35).
2. `CapteurLuminosite` : convertit la lecture en pourcentage (0-100%).

Questions :

- Que signifie le mot-clé `virtual` ?
- Que signifie `protected` et en quoi diffère-t-il de `private` ?
- Quel est l'avantage de l'héritage ici ?

Exercice 14. Composition : Système d'alarme

Créer une bibliothèque `Alarme` qui **compose** plusieurs objets existants.

Spécifications :

- L'alarme utilise : 1 Bouton (armement), 1 `CapteurAnalogique` (détection), 1 LED (état).
- États : DESARMEE, ARMEE, DECLENCHEE.
- Temporisation de 10 secondes pour quitter la zone après armement.
- Seuil de détection configurable.

Travail demandé :

1. Réutiliser les bibliothèques créées précédemment (composition).
2. Implémenter une machine à états pour gérer les transitions.
3. Le `.h` doit inclure les autres bibliothèques nécessaires.

```
1 #include "Bouton.h"
2 #include "CapteurAnalogique.h"
3 #include "LED.h"
4
5 class Alarme {
6 public:
7     Alarme(int pinBouton, int pinCapteur, int pinLED, int seuil);
8     void actualiser();
9     int getEtat();
10 private:
```

```

11   Bouton _bouton;
12   CapteurAnalogique _capteur;
13   LED _led;
14   int _etat;
15   unsigned long _tempoDepart;
16   // ...
17 };

```

Exercice 15. Callbacks : Bibliothèque Timer générique

Créer une bibliothèque **Timer** qui exécute une fonction utilisateur à intervalles réguliers.

Spécifications :

- Le constructeur reçoit l'intervalle en ms et un pointeur de fonction (callback).
- Méthodes : `demarrer()`, `arreter()`, `setIntervalle()`.
- La méthode `actualiser()` vérifie si l'intervalle est écoulé et appelle le callback.

Travail demandé :

1. Comprendre la syntaxe des pointeurs de fonction en C++.
2. Implémenter la bibliothèque.
3. Écrire un exemple avec deux timers indépendants.

```

1 // Type pour le pointeur de fonction
2 typedef void (*CallbackFunction)();
3
4 class Timer {
5 public:
6     Timer(unsigned long intervalle, CallbackFunction callback);
7     void demarrer();
8     void arreter();
9     void actualiser();
10 private:
11     unsigned long _intervalle;
12     unsigned long _dernierAppel;
13     CallbackFunction _callback;
14     bool _actif;
15 };

```

Exemple d'utilisation :

```

1 void clignoterLED() {
2     digitalWrite(13, !digitalRead(13));
3 }
4
5 Timer monTimer(500, clignoterLED);
6
7 void setup() {
8     pinMode(13, OUTPUT);
9     monTimer.demarrer();
10 }
11
12 void loop() {
13     monTimer.actualiser();
14 }

```


Exercice 16. Bibliothèque avec templates

Créer une bibliothèque `PileGenerique` (Stack) qui fonctionne avec n'importe quel type de données.

Spécifications :

- Pile LIFO de capacité fixe (template parameter).
- Méthodes : `empiler(T valeur)`, `depiler()`, `sommet()`, `estVide()`, `estPleine()`.
- Doit fonctionner avec `int`, `float`, `byte`, ou même des structures.

Travail demandé :

1. Écrire la classe template (tout dans le `.h` car les templates ne se séparent pas).
2. Tester avec différents types de données.

```
1 template <typename T, int CAPACITE>
2 class PileGenerique {
3 public:
4     PileGenerique() : _sommet(-1) {}
5
6     bool empiler(T valeur) {
7         if (_sommet >= CAPACITE - 1) return false;
8         _donnees[++_sommet] = valeur;
9         return true;
10    }
11
12    T depiler() {
13        return _donnees[_sommet--];
14    }
15
16    // ... autres methodes
17
18 private:
19     T _donnees[CAPACITE];
20     int _sommet;
21 };
```

Question : Pourquoi les classes templates doivent-elles être entièrement définies dans le `.h` ?

Partie V : Modification de Bibliothèques Existantes

Exercice 17. Étendre la bibliothèque Servo

La bibliothèque standard `Servo` ne permet pas de connaître la position actuelle du servo sans la mémoriser soi-même.

Travail demandé :

1. Créer une classe `ServoEtendu` qui hérite de `Servo`.
2. Ajouter une méthode `getPosition()` qui retourne le dernier angle envoyé.
3. Ajouter une méthode `bougerVers(int angle, int vitesse)` qui déplace progressivement le servo.

```

1 #include <Servo.h>
2
3 class ServoEtendu : public Servo {
4 public:
5     void ecrire(int angle);           // Surcharge pour memoriser
6     int getPosition();
7     void bougerVers(int angle, int delaiMs);
8     void actualiser();               // Pour mouvement progressif
9 private:
10    int _positionActuelle;
11    int _positionCible;
12    int _delai;
13    unsigned long _dernierMouvement;
14 };

```

Exercice 18. Wrapper pour bibliothèque tierce

On utilise une bibliothèque DHT pour lire un capteur de température/humidité, mais son interface n'est pas pratique.

Travail demandé : Créer une classe `CapteurDHT` qui encapsule la bibliothèque DHT et offre :

1. Une lecture avec cache (ne relire le capteur que toutes les 2 secondes).
2. Des méthodes séparées `getTemperature()` et `getHumidite()`.
3. Une méthode `getIndiceConfort()` qui calcule un indice basé sur les deux valeurs.
4. Gestion des erreurs de lecture (valeur NaN).

```

1 #include <DHT.h>
2
3 class CapteurDHT {
4 public:
5     CapteurDHT(int broche, int type);
6     void actualiser();
7     float getTemperature();
8     float getHumidite();
9     float getIndiceConfort();
10    bool estValide();
11 private:
12    DHT _dht; // Composition : on encapsule l'objet DHT
13    float _temperature, _humidite;
14    unsigned long _derniereLecture;
15    bool _valide;
16 };

```

Exercice 19. Adaptation d'une bibliothèque pour économiser la RAM

Une bibliothèque tierce `GrandBuffer` utilise un buffer de 512 octets, ce qui est trop pour votre projet.

Scénario :

```

1 // Fichier original : GrandBuffer.h
2 class GrandBuffer {
3     // ...

```

```

4 private:
5     byte _buffer[512]; // Trop gros !
6 };

```

Travail demandé :

1. Sans modifier la bibliothèque originale, créer une classe `PetitBuffer` qui hérite de `GrandBuffer` et remplace le buffer (si possible).
2. Si l'héritage n'est pas possible, proposer une solution par copie et modification.
3. Discuter des risques de modifier une bibliothèque tierce.

Question : Comment gérer les mises à jour de la bibliothèque originale ?

Partie VI : Projets Intégrateurs

Exercice 20. Bibliothèque Menu pour écran LCD

Créer une bibliothèque complète pour gérer un menu hiérarchique sur un écran LCD 16×2.

Spécifications :

- Structure de menu en arbre (éléments, sous-menus, actions).
- Navigation avec 3 boutons : Haut, Bas, Valider.
- Affichage avec curseur de sélection.
- Possibilité d'associer une fonction callback à chaque élément.

Travail demandé :

1. Définir une structure `ElementMenu` (label, type, callback, sous-menu).
2. Stocker les labels en `PROGMEM` pour économiser la RAM.
3. Implémenter la navigation dans l'arbre.

```

1 typedef void (*ActionMenu)();
2
3 struct ElementMenu {
4     const char* label;           // Pointeur vers PROGMEM
5     byte type;                   // 0=action, 1=sous-menu
6     ActionMenu action;           // Fonction à appeler
7     ElementMenu* sousMenu;       // Pointeur vers sous-menu
8     byte nbElements;             // Nombre d'elements du sous-menu
9 };
10
11 class Menu {
12 public:
13     Menu(LiquidCrystal* lcd, ElementMenu* racine, int nbElements);
14     void afficher();
15     void haut();
16     void bas();
17     void valider();
18     void retour();
19 private:
20     // ...
21 };

```

Exercice 21. Bibliothèque DataLogger avec stockage EEPROM

Créer une bibliothèque pour enregistrer des données de capteurs en EEPROM avec gestion circulaire.

Spécifications :

- Enregistrement horodaté de valeurs (timestamp + valeur).
- Gestion circulaire : quand l'EEPROM est pleine, on écrase les données les plus anciennes.
- Méthodes d'accès : `getNbEnregistrements()`, `getEnregistrement(int i)`.
- Optimisation : minimiser les écritures EEPROM (durée de vie limitée).

Travail demandé :

1. Calculer combien d'enregistrements tiennent dans 1 Ko (structure de 6 octets).
2. Implémenter l'écriture circulaire avec pointeur de tête.
3. Stocker le pointeur de tête à une adresse fixe.
4. Utiliser `EEPROM.update()` pour économiser les cycles d'écriture.

```
1 struct Enregistrement {
2     unsigned long timestamp; // 4 octets
3     int valeur; // 2 octets
4 };
5
6 class DataLogger {
7 public:
8     DataLogger(int adresseDebut, int tailleMax);
9     void enregistrer(int valeur);
10    int getNbEnregistrements();
11    Enregistrement getEnregistrement(int index);
12    void effacerTout();
13 private:
14     int _adresseDebut;
15     int _tailleMax;
16     int _tete;
17     int _nbEnregistrements;
18     void _sauvegarderMetadonnees();
19 };
```

Exercice 22. Bibliothèque Communication (Protocole maître-esclave)

Créer une bibliothèque pour faire communiquer deux Arduino via le port série avec un protocole structuré.

Spécifications du protocole :

- Trame : [START] [TYPE] [LONGUEUR] [DONNEES...] [CHECKSUM]
- START = 0xAA, TYPE = 1 octet, LONGUEUR = 1 octet, CHECKSUM = XOR de tous les octets.
- Gestion des timeouts et des erreurs de transmission.

Travail demandé :

1. Implémenter `envoyerTrame(byte type, byte* donnees, byte longueur)`.
2. Implémenter `recevoirTrame()` avec machine à états.
3. Gérer un buffer de réception et la validation du checksum.

```
1 class ProtocoleSerie {
2 public:
3     ProtocoleSerie(HardwareSerial* serial);
4     void envoyerTrame(byte type, byte* donnees, byte longueur);
5     bool trameDispo();
6     byte getTypeTrame();
7     byte* getDonnees();
8     byte getLongueur();
9     void actualiser(); // Machine a etats de reception
10 private:
11     HardwareSerial* _serial;
12     byte _bufferRx[64];
13     byte _etatReception;
14     byte _indexBuffer;
15     unsigned long _dernierOctet;
16     // ...
17 };
```

Questions de Synthèse

Question A : Architecture logicielle

Pour un projet complexe (robot mobile avec capteurs, moteurs, écran, communication), comment organiseriez-vous vos bibliothèques ?

1. Proposer une hiérarchie de classes.
2. Identifier les dépendances entre bibliothèques.
3. Discuter du compromis entre modularité et consommation mémoire.

Question B : Bonnes pratiques

Lister les bonnes pratiques à respecter lors de la création d'une bibliothèque Arduino :

- Nommage des fichiers et des classes
- Documentation (commentaires, exemples)
- Gestion des erreurs
- Compatibilité avec différentes cartes
- Tests et validation

Question C : Optimisation mémoire

Pour chaque technique, expliquer son impact sur la mémoire :

1. Utiliser `byte` au lieu de `int` pour les petites valeurs
2. Stocker les chaînes constantes en `PROGMEM`

3. Préférer les tableaux de taille fixe aux allocations dynamiques
4. Utiliser des références (&) plutôt que des copies pour les gros objets

Bon courage !